Asino Fredrick 225098636

Problem: Online Store Challenges

BrightMart is an online store that sells electronics and groceries. The store owner, the staff who process orders, and the customers are all affected by the problems below.

The problem: Without a structured system, the store handles electronics and groceries under the same rules, although they are not the same. Groceries spoil and need expiry tracking. Electronics don't, but they need warranty tracking. Stock is not tracked per product, so the store can sell items that are out of stock or expired.

Payments are also inconsistent. A credit card and a mobile money payment need different validation checks, and nothing enforces them in one place. For example, a mobile money payment can be accepted with an invalid phone number.

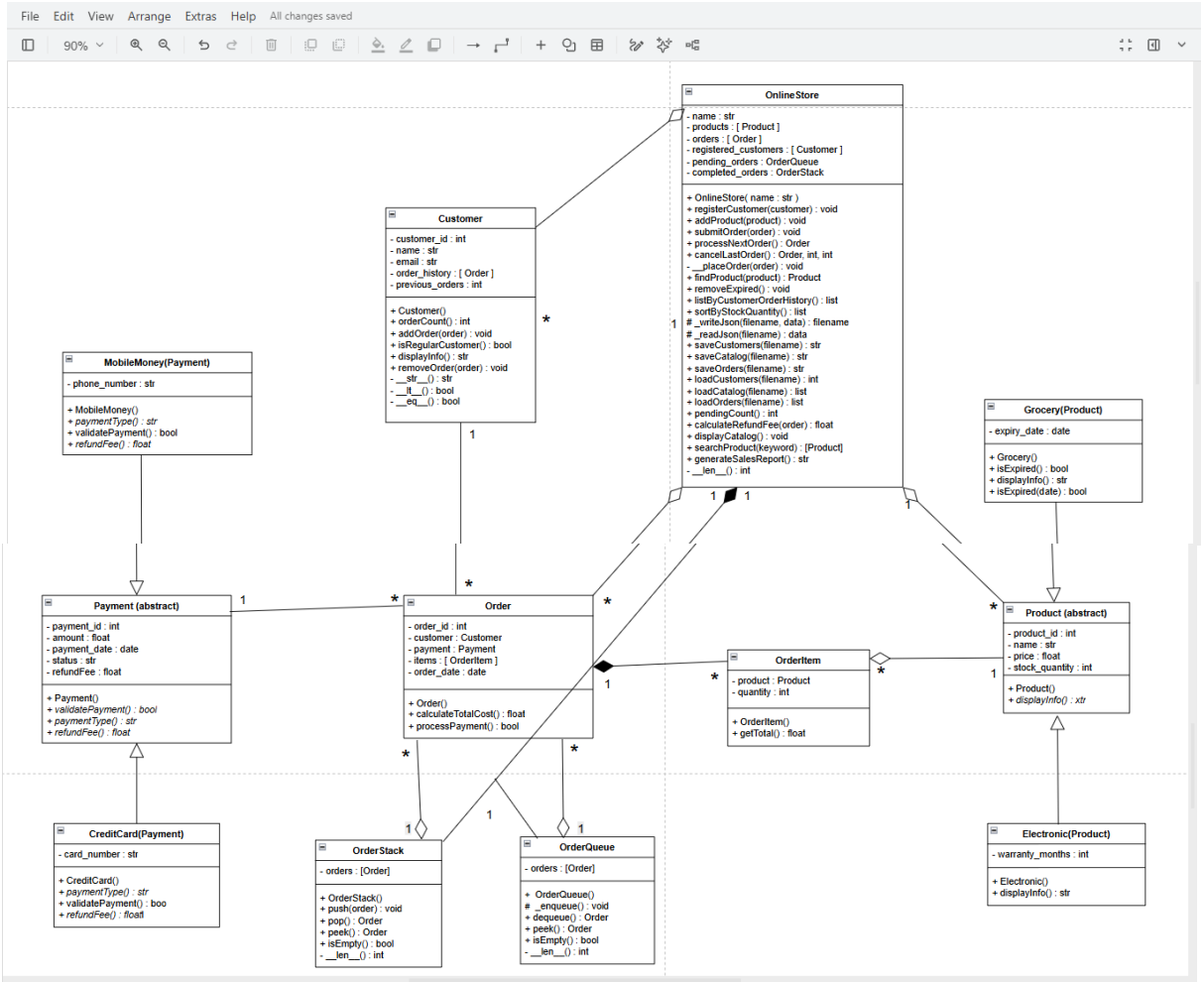
Loyal customers are treated like first-time buyers because nothing tracks order history to apply a discount.

Three more problems appear as the store grows:

1. **Orders pile up with no fixed handling order.** Orders arrive faster than they can be checked and paid for, so a customer who ordered first can be served last.
2. **Placed orders can't be undone.** When a customer cancels or an order was entered by mistake, the stock is not returned, the sale still counts in the sales figures, and the refund fee that depends on how the customer paid (5% on credit card, none on mobile money) is not applied. Staff have to fix the records by hand.
3. **Data is lost when the program closes.** Orders and sales figures are not kept, so the owner can't review them later.
4. **Data has to be typed in by hand.** The product catalog and customer details must be re-entered each time the program starts.

Invalid input, such as a negative quantity, an expired product or a wrong phone number, also stops the program instead of being reported to the user. Staff who aren't programmers need to operate all of this without using code.

The result: overselling expired or out-of-stock items, inconsistent payment handling, unfair order processing, lost records, repeated manual entry, and no reward for repeat customers.


File Edit View Arrange Extras Help All changes saved
90%
OnlineStore
- name : str
- products : [Product]
- orders : [Order]
- registered_customers : [Customer]
- pending_orders : OrderQueue
- completed_orders : OrderStack
+ OnlineStore(name : str)
+ registerCustomer(customer) : void
+ addProduct(product) : void
+ submitOrder(order) : void
+ processNextOrder() : Order
+ cancelLastOrder() : Order, int, int
- __placeOrder(order) : void
+ findProduct(product) : Product
+ removeExpired() : void
+ listByCustomerOrderHistory() : list
+ sortByStockQuantity() : list
_writeJson(filename, data) : filename
_readJson(filename) : data
+ saveCustomers(filename) : str
+ saveCatalog(filename) : str
+ saveOrders(filename) : str
+ loadCustomers(filename) : int
+ loadCatalog(filename) : list
+ loadOrders(filename) : list
+ pendingCount() : int
+ calculateRefundFee(order) : float
+ displayCatalog() : void
+ searchProduct(keyword) : [Product]
+ generateSalesReport() : str
- __len__() : int
Customer
- customer_id : int
- name : str
- email : str
- order_history : [Order]
- previous_orders : int
+ Customer()
+ orderCount() : int
+ addOrder(order) : void
+ isRegularCustomer() : bool
+ displayInfo() : str
+ removeOrder(order) : void
- __str__() : str
- __lt__() : bool
- __eq__() : bool
MobileMoney(Payment)
- phone_number : str
+ MobileMoney()
+ paymentType() : str
+ validatePayment() : bool
+ refundFee() : float
Grocery(Product)
- expiry_date : date
+ Grocery()
+ isExpired() : bool
+ displayInfo() : str
+ isExpired(date) : bool
Payment (abstract)
- payment_id : int
- amount : float
- payment_date : date
- status : str
- refundFee : float
+ Payment()
+ validatePayment() : bool
+ paymentType() : str
+ refundFee() : float
Order
- order_id : int
- customer : Customer
- payment : Payment
- items : [OrderItem]
- order_date : date
+ Order()
+ calculateTotalCost() : float
+ processPayment() : bool
OrderItem
- product : Product
- quantity : int
+ OrderItem()
+ getTotal() : float
Product (abstract)
- product_id : int
- name : str
- price : float
- stock_quantity : int
+ Product()
+ displayInfo() : str
CreditCard(Payment)
- card_number : str
+ CreditCard()
+ paymentType() : str
+ validatePayment() : boo
+ refundFee() : float
OrderStack
- orders : [Order]
+ OrderStack()
+ push(order) : void
+ pop() : Order
+ peek() : Order
+ isEmpty() : bool
- __len__() : int
OrderQueue
- orders : [Order]
+ OrderQueue()
_enqueue() : void
+ dequeue() : Order
+ peek() : Order
+ isEmpty() : bool
- __len__() : int
Electronic(Product)
- warranty_months : int
+ Electronic()
+ displayInfo() : str